WEEKLY MENTEE DSA CHALLENGE

9 Problems
Arrays & Problem Solving

Practice • Analyze • Explain

Mentee name: _____	**Submission date:** _____
College / Roll No.: _____	**Language:** _____

Instructions

1. Solve all 9 problems independently.

2. For each problem, write the approach and explain why it works.

3. Include time and space complexity (Big-O).

4. Test the given examples and at least one additional edge case.

5. Submit your code in the language assigned or approved by your mentor.

6. Be prepared to explain your solution and dry-run it during the review.

Mentor review focus: correctness • approach • complexity • edge cases • clear explanation

Problem Set

Read each problem carefully. The examples are included to clarify the expected output; they are not the only test cases.

1. Majority Element I

Focus: Arrays • Frequency counting

Given an integer array **nums** of size **n**, return its majority element. The majority element appears more than **n / 2** times. The array is guaranteed to contain a majority element.

Example 1

Input: nums = [7, 0, 0, 1, 7, 7, 2, 7, 7]
Output: 7
Explanation: 7 appears 5 times in an array of size 9.

Example 2

Input: nums = [1, 1, 1, 2, 1, 2]
Output: 1
Explanation: 1 appears 4 times in an array of size 6.

My approach / complexity: _____

2. Leaders in an Array

Focus: Arrays • Traversal from right

Given an integer array **nums**, return all leaders in the array. A leader is an element strictly greater than every element to its right. The rightmost element is always a leader. Return leaders in the same order in which they appear in **nums**.

Example 1

Input: nums = [1, 2, 5, 3, 1, 2]
Output: [5, 3, 2]
Explanation: 5, 3 and 2 are greater than every element to their right.

Example 2

Input: nums = [-3, 4, 5, 1, -4, -5]
Output: [5, 1, -4, -5]
Explanation: Each listed value is strictly greater than all values to its right.

My approach / complexity: _____

3. Rearrange Array Elements by Sign

Focus: Arrays • Two pointers / indexing

Given an even-length array with an equal number of positive and negative integers, rearrange it so that consecutive elements have opposite signs, the result starts with a positive integer, and the relative order of positive values and of negative values is preserved.

Example 1

Input: nums = [2, 4, 5, -1, -3, -4]
Output: [2, -1, 4, -3, 5, -4]
Explanation: Positive and negative values each keep their original relative order.

Example 2

Input: nums = [1, -1, -3, -4, 2, 3]
Output: [1, -1, 2, -3, 3, -4]
Explanation: The output alternates signs and begins with a positive value.

My approach / complexity: _____

4. Pascal's Triangle I

Focus: Arrays • Combinatorics

Given two integers **r** and **c**, return the value at row **r** and column **c** of Pascal's Triangle. Rows and columns are **1-indexed**. Each row starts and ends with 1; each interior value is the sum of the two values directly above it.

Example 1

Input: r = 4, c = 2
Output: 3
Explanation: Row 4 is [1, 3, 3, 1], so column 2 contains 3.

Example 2

Input: r = 5, c = 3
Output: 6
Explanation: Row 5 is [1, 4, 6, 4, 1], so column 3 contains 6.

My approach / complexity: _____

5. Sort an Array of 0s, 1s and 2s

Focus: Arrays • In-place sorting

Given an array **nums** containing only 0, 1 and 2, sort it in non-decreasing order. The sorting must be done in-place, without creating a copy of the original array.

Example 1

Input: nums = [1, 0, 2, 1, 0]
Output: [0, 0, 1, 1, 2]
Explanation: There are two 0s, two 1s and one 2.

Example 2

Input: nums = [0, 0, 1, 1, 1]
Output: [0, 0, 1, 1, 1]
Explanation: The array is already in non-decreasing order.

My approach / complexity: _____

6. Kadane's Algorithm — Maximum Subarray Sum

Focus: Arrays • Dynamic programming

Given an integer array **nums**, find the non-empty contiguous subarray with the largest sum and return that sum.

Example 1

Input: nums = [2, 3, 5, -2, 7, -4]
Output: 15
Explanation: The subarray [2, 3, 5, -2, 7] has sum 15.

Example 2

Input: nums = [-2, -3, -7, -2, -10, -4]
Output: -2
Explanation: The maximum sum is the largest individual element, -2.

My approach / complexity: _____

7. Merge Two Sorted Arrays Without Extra Space

Focus: Arrays • Two pointers

Given two sorted integer arrays **nums1** and **nums2**, merge them into sorted order in-place. **nums1** has length **m + n**: its first **m** elements are valid values and its remaining **n** positions are placeholders. **nums2** has length **n**. Store the final merged array in **nums1**.

Example 1

Input: nums1 = [-5, -2, 4, 5, 0, 0, 0]; nums2 = [-3, 1, 8]
Output: [-5, -3, -2, 1, 4, 5, 8]
Explanation: The final sorted array is stored in nums1.

Example 2

Input: nums1 = [0, 2, 7, 8, 0, 0, 0]; nums2 = [-7, -3, -1]
Output: [-7, -3, -1, 0, 2, 7, 8]
Explanation: Merge the valid elements from both arrays.

My approach / complexity: _____

8. Count Inversions

Focus: Arrays • Merge sort

Given an integer array **nums**, return its inversion count. A pair of indices **(i, j)** is an inversion when **i < j** and **nums[i] > nums[j]**. A sorted array has zero inversions.

Example 1

Input: nums = [2, 3, 7, 1, 3, 5]
Output: 5
Explanation: The inversions are (2,1), (3,1), (7,1), (7,3) and (7,5), using their original index positions.

Example 2

Input: nums = [-10, -5, 6, 11, 15, 17]
Output: 0
Explanation: The array is already sorted, so it has no inversions.

My approach / complexity: _____

9. Reverse Pairs

Focus: Arrays • Merge sort

Given an integer array **nums**, count the pairs of indices **(i, j)** satisfying **0 ≤ i < j < nums.length** and **nums[i] > 2 × nums[j]**.

Example 1

Input: nums = [6, 4, 1, 2, 7]
Output: 3
Explanation: The reverse pairs are (0,2), (0,3) and (1,2).

Example 2

Input: nums = [5, 4, 4, 3, 3]
Output: 0
Explanation: No pair satisfies nums[i] > 2 × nums[j].

My approach / complexity: _____

Final Submission Checklist

■ All 9 solutions are submitted.

- ■ Each solution includes time and space complexity.
- ■ Examples and edge cases have been tested.
- ■ I can explain and dry-run each solution without copying.